Polynomial Interpolation - Part 2

Alexandra A. Gomes

April 15, 2026

First Interpolation Error Theorem

Let $x_0, \ldots x_n$ be $n+1$ **distinct** nodes in $[a, b]$, $f \in C^{n+1}[a, b]$, and p_n the interpolating polynomial of degree $\leq n$ of f at these nodes. Then, for each $x \in [a, b]$, there exists a number $\xi(x)$ between the nodes, that is, in (a, b) such that

$$f(x) = p_n(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)\ldots(x - x_n).$$

Equally Spaced Nodes: Interpolation Error Theorem

The interpolation error, $e_n(x) = f(x) - p_n(x)$, verifies

$$|e_n(x)| \leq \frac{M}{4\,(n+1)} h^{n+1}.$$

Practicalities

In general, we do not have f or its derivatives (in data fitting). Is there a way to get a measure of the interpolation error?

- calculate interpolating polynomials, p_{n-1}, p_n, p_{n+1} using different subsets of the same set of nodes and compare results;
- divide the data set into 2 sets: use one set to construct the interpolant and the other to evaluate its accuracy.

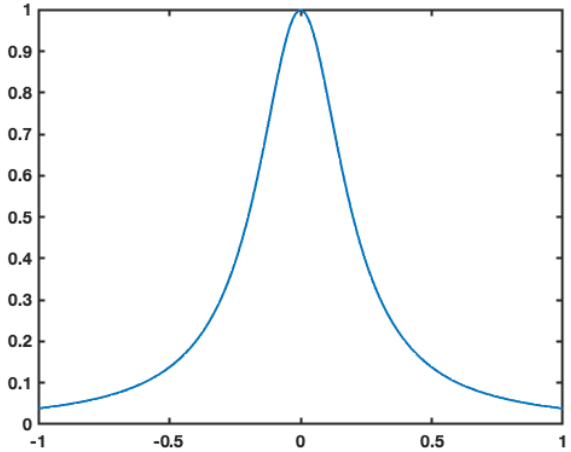
Interpolation Error

$$|e_n(x)| \leq \frac{M}{4\,(n+1)} h^{n+1}$$

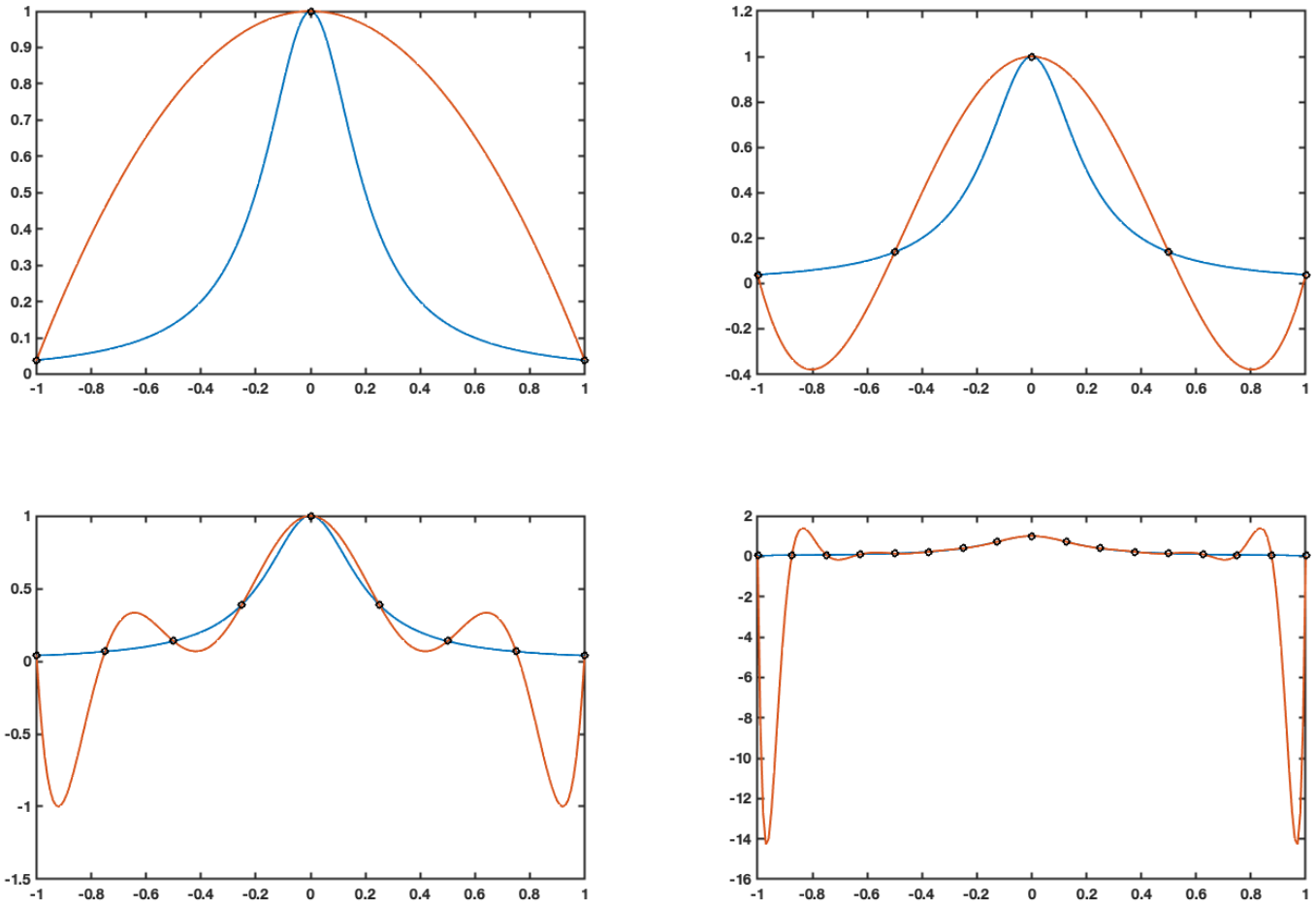
1. What is the error involved in approximating a function by an interpolating poly?
2. Is f well approximated at intermediate points?
3. Will more nodes increase the quality of the interpolating polynomial?
4. Can we choose the location of the nodes to increase the quality of the interpolating polynomial?

Runge Function

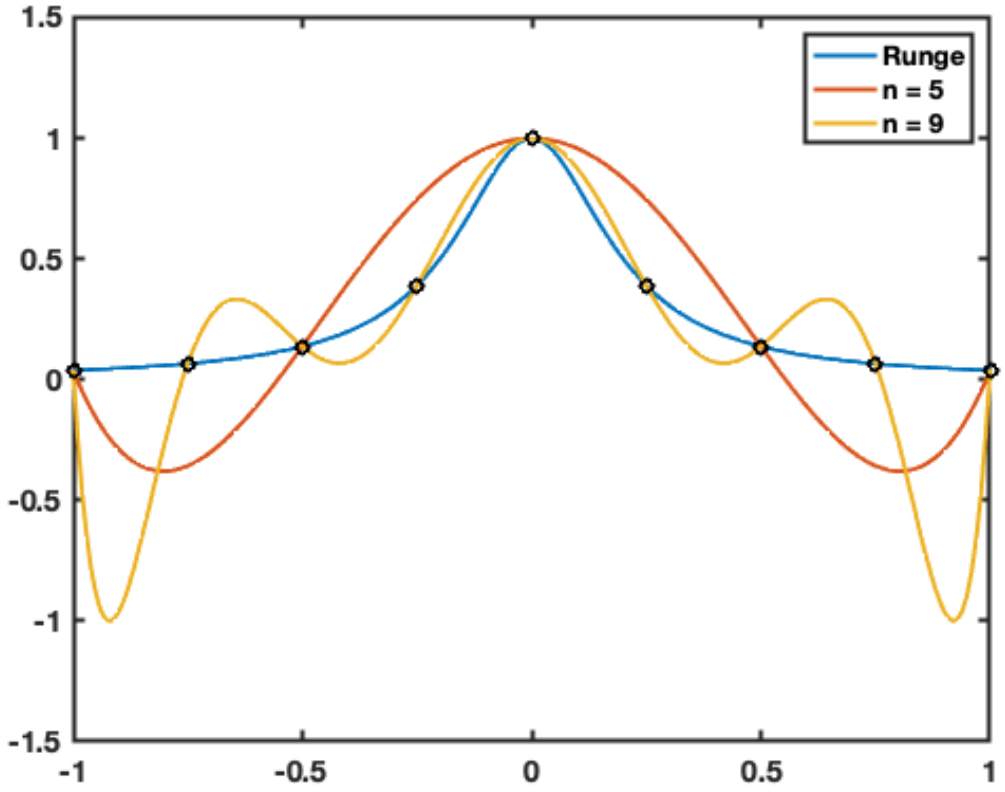
$$f(x) = \frac{1}{1 + 25x^2}, \qquad -1 \leq x \leq 1$$



Runge Phenomenon: Different Interpolants



Runge Phenomenon: Different Interpolants



- the approximation of f gets worse as n increases;
- the interpolation error is larger near the boundaries of the interval of interest.

Runge Phenomenon

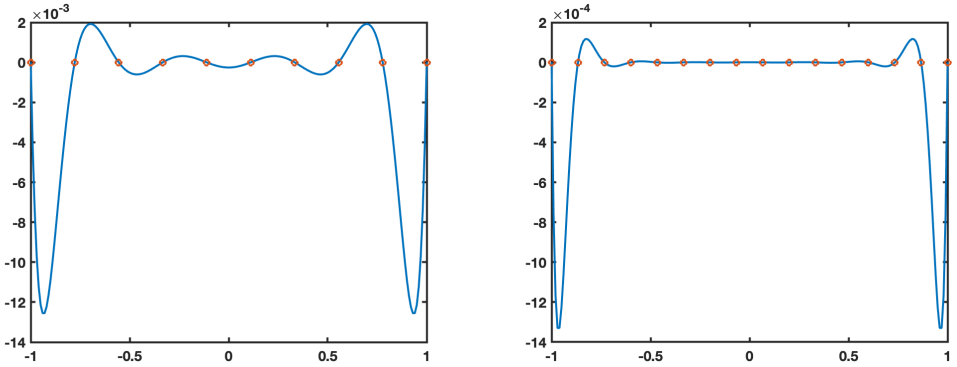
$$f(x) = p_n(x) + \frac{f^{(n+1)}(\xi(x))}{(n+1)!}(x - x_0)\dots(x - x_n);$$

$$|e_n(x)| \leq \frac{M}{4(n+1)} h^{n+1};$$

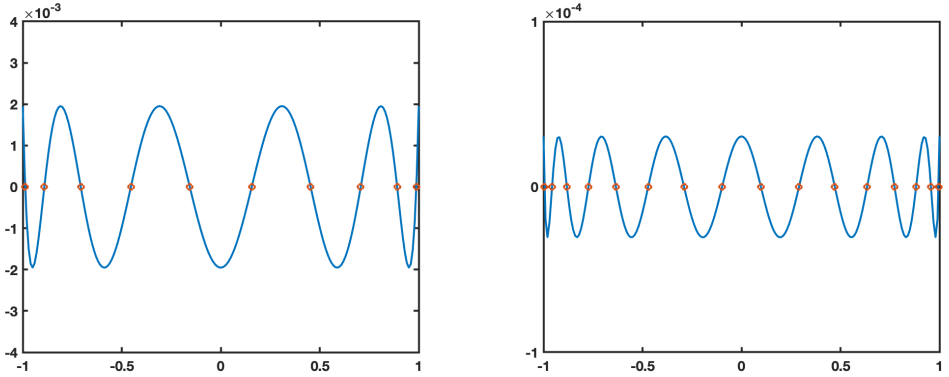
- M is an upper bound of the $n+1$ derivative;
- but the value of the derivative, $f^{(n+1)}(\xi(x))$, changes with x and n;
- for the Runge function, $f^{(n+1)}(x) / (n+1)!$ grows with n near the endpoints of the interval;
- as such, the interpolation error increases with n;
- but the fundamental problem is that we have an *equidistant distribution* of points.

Graph of $\prod_{i=0}^{n}(x - x_i)$

- equally spaced points, $n + 1 = 10, 16$:



- Chebyshev points, $n + 1 = 10, 16$:



Runge Phenomenon: Solution

- do not use an equidistant distribution of nodes;
- it is usually not a good idea to use uniformly spaced interpolation points, unless the degree of the polynomial is low;
- instead, use systems of nodes clustered at the boundary;
- this not a problem of the Lagrange interpolating polys: any other basis will have the same problem since the interpolant is unique;

Chebyshev Points

- choose x_i such that

$$\min_{x_i} \max_{a \leq x \leq b} \prod_{i=0}^{n} (x - x_i);$$

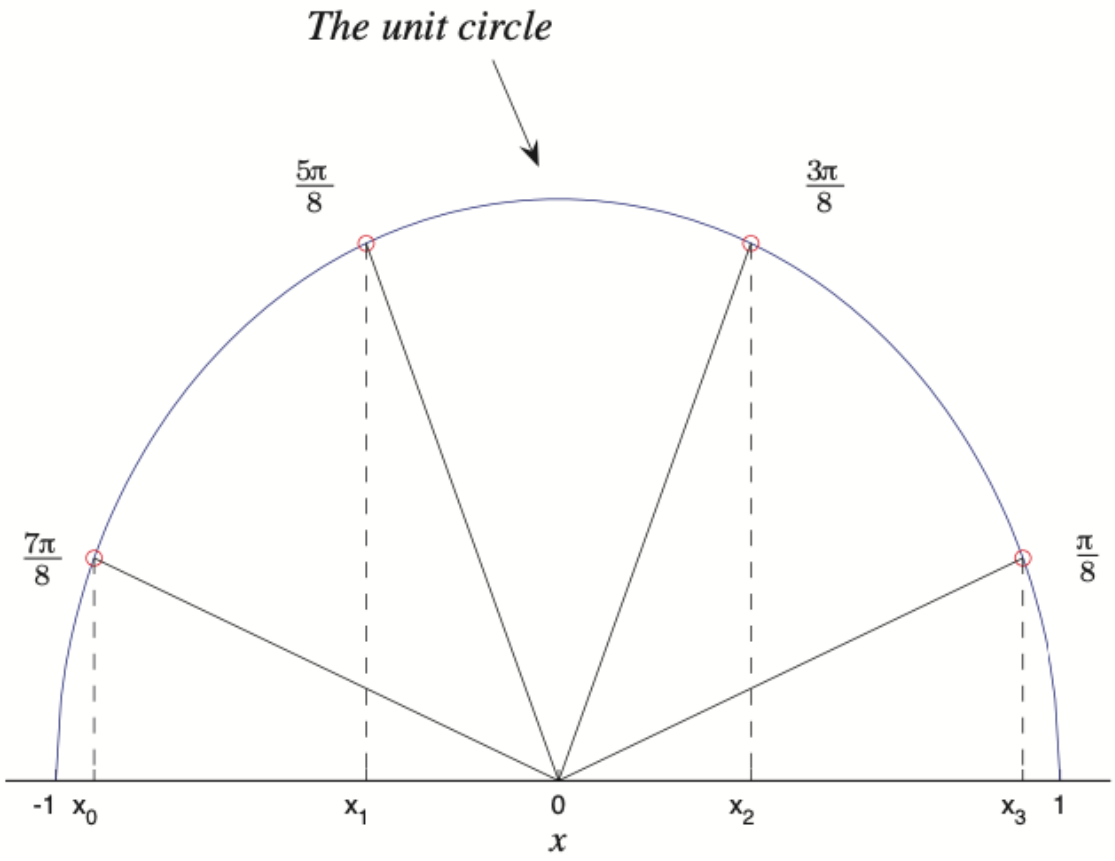
- Chebyshev points have a simple closed form formula in the interval $[-1, 1]$:

$$x_i = \cos\left(\frac{2i+1}{2(n+1)}\pi\right), \quad i = 0, \ldots, n;$$

- and in a general interval $[a, b]$, we scale and translate $[-1, 1]$ onto $[a, b]$

$$x_i \leftarrow a + \frac{b-a}{2}(x_i + 1), \quad i = 0, \ldots, n.$$

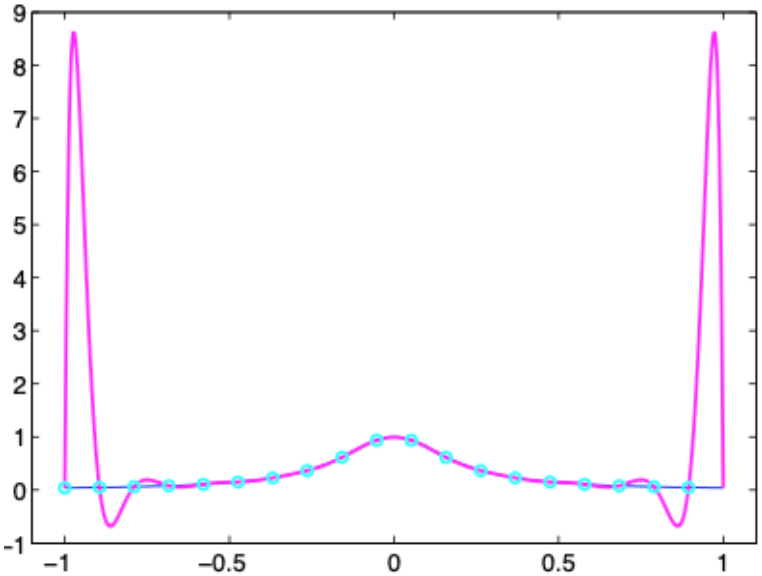
Chebyshev Points

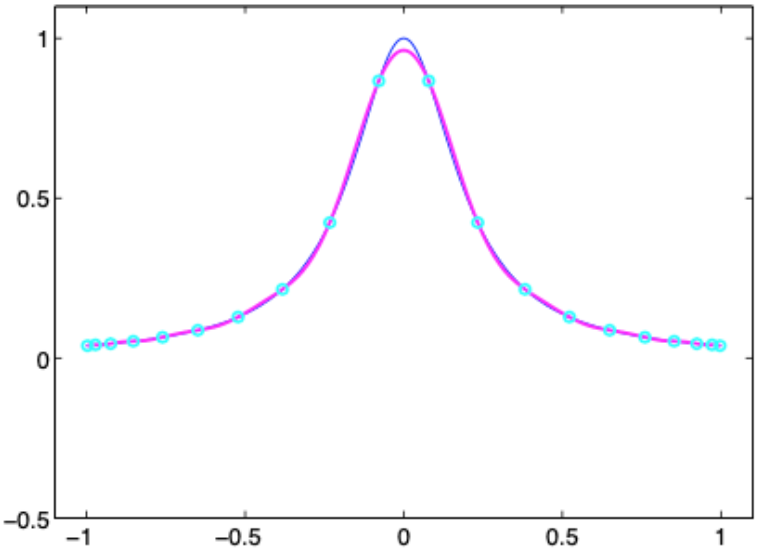


- Chebyshev points cluster at the boundaries
- these nodes are the x-coordinates of equally spaced points on a unit circle.

Runge Function Example

- compare the $n = 20$ interpolant for equidistant and Chebyshev points:





Taylor Approximation vs Interpolant Approximation

- for Taylor approximation:
  1. f can be approximated by a degree n polynomial if we can compute $f(x_0)$ and derivatives of f up to degree n at x_0;
  2. we only need information at a *single point*: x_0;
- for interpolant:
  - the interpolant approximation is also a degree n polynomial, computed from function values at $n+1$ points;
  - needs information from $n+1$ points.

Newton's Interpolant

- the Lagrange's form of the interpolating polynomial of f is

$$p_n(x) = \sum_{i=0}^{n} f(x_i) L_{n,i}(x),$$

  in which $L_{n,i}(x)$ are the Lagrange interpolating polynomials;
- p_n is unique but has different algebraic representations, each more convenient for certain applications;
- we can express p_n in Newton's form as

$$p_n(x) = a_0 + a_1(x - x_0) + \ldots + a_n(x - x_0)\ldots(x - x_{n-1})$$

  for constants $a_0, \ldots a_n$, which we need to determine.

Newton's Interpolant

$$p_n(x) = a_0 + a_1(x - x_0) + \ldots + a_n(x - x_0)\ldots(x - x_{n-1})$$

- $$p_n(x) = \sum_{i=0}^{n} a_i \prod_{j=0}^{i-1}(x - x_j),$$

  in which $\prod_{j=0}^{i-1}(x - x_j)$ are the elements of Newton polynomial basis;
- the advantage is that we can easily add more data points.

Divided Differences

$$p_n(x) = a_0 + a_1(x - x_0) + \ldots + a_n(x - x_0)\ldots(x - x_{n-1})$$

- $a_0 = p_n(x_0) = f(x_0)$
- $f(x_0) + a_1(x_1 - x_0) = p_n(x_1) = f(x_1) \Rightarrow a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}$

Newton Divided Differences

- $$f[x_i] = f(x_i)$$
  is the **zeroth divided difference** of f w.r.t x_i;
- $$f[x_i, x_{i+1}] = \frac{f[x_{i+1}] - f[x_i]}{x_{i+1} - x_i}$$
  is the **first divided difference** of f w.r.t x_i and x_{i+1};
- $$f[x_i, x_{i+1}, x_{i+2}] = \frac{f[x_{i+1}, x_{i+2}] - f[x_{i+1}, x_i]}{x_{i+2} - x_i}$$
  is the **second divided difference** of f w.r.t x_i, x_{i+1} and x_{i+2};
- $$f[x_i, x_{i+1}, \ldots, x_{i+k-1}, x_{i+k}] =$$
  $$= \frac{f[x_{i+1}, x_{i+2}, \ldots, x_{i+k}] - f[x_i, x_{i+1}, \ldots, x_{i+k-1}]}{x_{i+k} - x_i}$$
  is the **kth divided difference** of f

Newton Divided Differences

- $$f\left[x_0, x_1, \ldots, x_{n-1}, x_n\right] = \\ = \frac{f\left[x_1, x_2, \ldots, x_n\right] - f\left[x_0, x_1, \ldots, x_{n-1}\right]}{x_n - x_0}$$

  is the **nth divided difference** of f and it is a recursive formula for the divided differences;

then:

$$p_n(x) = a_0 + a_1(x - x_0) + \ldots + a_n(x - x_0)\ldots(x - x_{n-1})$$

$$\begin{aligned} p_n(x) = f\left[x_0\right] \quad &+ \quad f\left[x_0, x_1\right](x - x_0) + \ldots + \\ &+ \quad f\left[x_0, \ldots, x_n\right](x - x_0)\ldots(x - x_{n-1}) \end{aligned}$$

$$p_n(x) = f\left[x_0\right] + \sum_{k=1}^{n} f\left[x_0, \ldots, x_k\right](x - x_0)\ldots(x - x_{k-1})$$

Divided Differences Table

i	x_i	$f[x_i]$	$f[x_{i-1},x_i]$	$f[x_{i-2},x_{i-1},x_i]$	$\cdots$	$f[x_{i-n},\ldots,x_i]$
0	x_0	$f(x_0)$				
1	x_1	$f(x_1)$	$\frac{f[x_1]-f[x_0]}{x_1-x_0}$			
2	x_2	$f(x_2)$	$\frac{f[x_2]-f[x_1]}{x_2-x_1}$	$f[x_0,x_1,x_2]$		
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	
n	x_n	$f(x_n)$	$\frac{f[x_n]-f[x_{n-1}]}{x_n-x_{n-1}}$	$f[x_{n-2},x_{n-1},x_n]$	$\cdots$	$f[x_0,x_1,\ldots,x_n]$

Figure: from Ascher and Greif

example: find the interpolant through $(1,1),(2,3),(4,3)$.

Newton's Interpolating Polynomial: Assessment

- adapts easily to more points;
- construction: $O(3n^2/2)$;
- evaluation: $O(2n)$.

Divided Differences and Derivatives

Theorem: Let $f \in C^n[a, b]$ and $x_0, x_1, \ldots x_n$ be $n+1$ distinct numbers in $[a, b]$. Then, there exists $\xi \in [a, b]$ such that

$$f\left[x_0, x_1, \ldots, x_n\right] = \frac{f^{(n)}(\xi)}{(n)!}.$$

Polynomial Interpolation Error

Theorem: If p_n interpolates $f \in C^{n+1}[a,b]$ at the points $x_0, \ldots x_n$ contained in $[a,b]$, then for each $x \in [a,b]$, there exists $\xi(x) \in [a,b]$ such that

$$\begin{aligned} f(x) &= p_n(x) + \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i) \\ &= p_n(x) + f\left[x_0, x_1, \ldots, x_n, x\right] \prod_{i=0}^{n} (x - x_i) \end{aligned}$$

Hermite Interpolation

Recall that for **polynomial interpolation**, we have

$$p(x_k) = f(x_k)$$

at all the nodes x_k.

In Hermite interpolation, we ask more: in addition to $H(x_k) = f(x_k)$ at all the nodes x_k,

$$H'(x_k) = f'(x_k)$$

at all the nodes x_k.

$H(x) \rightarrow$ Hermite polynomial

Basic Theorem for Hermite Interpolation

If f belongs to $C^1[a,b]$ and $x_0, \ldots, x_n$ in $[a,b]$ are distinct nodes, the **unique** polynomial of least degree agreeing with f and f' at the nodes is the Hermite polynomial of degree at most $2n+1$ given by

$$H_{2n+1}(x) = \sum_{j=0}^{n} f(x_j) H_{n,j}(x) + \sum_{j=0}^{n} f'(x_j) \hat{H}_{n,j}(x),$$

in which

$$H_{n,j}(x) = \left[1 - 2(x - x_j) L'_{n,j}(x_j)\right] L^2_{n,j}(x),$$

$$\hat{H}_{n,j}(x) = (x - x_j) L^2_{n,j}(x),$$

$L_{n,j}(x)$ denotes the j-th Lagrange polynomial of degree n.
Moreover, if f belongs to $C^{2n+2}[a,b]$,

$$f(x) = H_{2n+1}(x) + \frac{(x-x_0)^2 \ldots (x-x_n)^2}{(2n+2)!} f^{(2n+2)}(\xi(x)).$$

Why Hermite Interpolation?

Because sometimes we need to come up with an approximation for a function that we can use in $(-\infty, +\infty)$!

There are more: Laguerre, Legendre, Chebyshev polynomials!

Flaws of Polynomial Interpolation

Recall that both polynomial and Hermite interpolation approximate f, in a closed interval, with **one** polynomial.

- the error term in
$$\frac{f^{(n+1)}\left(\xi\left(x\right)\right)}{(n+1)!}$$
may be big;
- high-order polys may oscillate erratically: a small fluctuation in a tiny portion of $[a, b]$ can induce large oscillations over the entire $[a, b]$;

Piecewise Polynomial Interpolation: Basic Idea

Piecewise Polynomial Interpolation: divide the interval into subintervals and construct an approximating polynomial for each subinterval.

Basic Approaches

- **piecewise-linear interpolation**:
  - join points $(x_0, f(x_0)), (x_1, f(x_1)), \ldots (x_n, f(x_n))$ by straight lines;
  - in general, there is no differentiability at the endpoints of the subintervals, which implies the lack of smoothness of the approximating function
- **piecewise Hermite polynomial**:
  - for each subinterval we have 2 points (endpoints); therefore, $n = 1$ and we can construct a cubic Hermite poly, which is smoother than a piecewise linear poly;
  - but we need to know f' at the endpoints

Cubic Spline Interpolation

- cubic polynomials in each subinterval
- 4 constants to be determined for each cubic polynomial: $ax^3 + bx^2 + cx + d$
- use these constants to impose smoothness on the approximating polynomial on the interval

Cubic Spline Interpolant: Definition

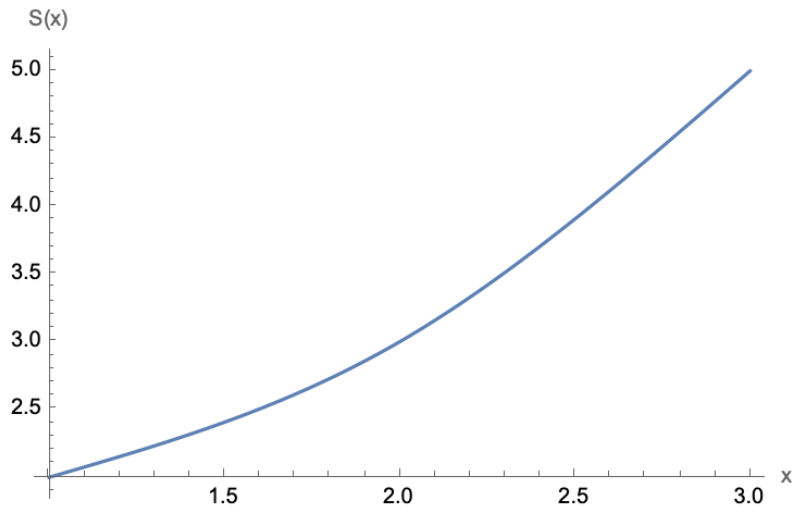
Consider a function f defined on $[a, b]$ and nodes $a = x_0 < x_1 < \ldots < x_n = b$. A **cubic spline interpolant** S for f is a function that satisfy the following conditions:

1. $S(x) = S_j(x)$, a cubic polynomial on $[x_j, x_{j+1}]$, for $j = 0, 1, \ldots, n-1$;
2. $S_j(x_j) = f(x_j)$ and $S_j(x_{j+1}) = f(x_{j+1})$, for $j = 0, 1, \ldots, n-1$, which implies $S_{j+1}(x_{j+1}) = S_j(x_{j+1})$, for $j = 0, 1, \ldots, n-2$;
3. $S'_{j+1}(x_{j+1}) = S'_j(x_{j+1})$, for $j = 0, 1, \ldots, n-2$;
4. $S''_{j+1}(x_{j+1}) = S''_j(x_{j+1})$, for $j = 0, 1, \ldots, n-2$;
5. choose one from two sets of boundary conditions:
   - 5.1 natural or free boundary: $S''(x_0) = S''(x_n) = 0$;
   - 5.2 clamped boundary: $S'(x_0) = f'(x_0)$ and $S'(x_n) = f'(x_n)$;

Cubic Spline Interpolant: Example

Natural cubic spline through $(1,2)$, $(2,3)$ and $(3,5)$?

$$S(x) = \begin{cases} 2 + \frac{3}{4}(x-1) + \frac{1}{4}(x-1)^3, x \in [1,2] \\ 3 + \frac{3}{2}(x-2) + \frac{3}{4}(x-2)^2 - \frac{1}{4}(x-2)^3, x \in [2,3] \end{cases}$$



Constructing the Cubic Spline

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$$

1. $S_j(x_j) = a_j$
2. $h_j = x_{j+1} - x_j$
3.

$$\begin{aligned} S_{j+1} \quad (x_{j+1}) &= a_{j+1} = S_j(x_{j+1}) = \\ &= a_j + b_j(x_{j+1} - x_j) + c_j(x_{j+1} - x_j)^2 + d_j(x_{j+1} - x_j)^3 \\ &= a_j + b_j h_j + c_j h_j^2 + d_j h_j^3 \end{aligned}$$

4. $S_j'(x_j) = b_j$
5. $b_{j+1} = b_j + 2c_j h_j + 3d_j h_j^2$
6. $\ldots$

Constructing the Cubic Spline

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3$$

1. ...
2. $b_{j+1} = b_j + h_j(c_j + c_{j+1})$
3. $c_{j+1} = c_j + 3d_j h_j$

$$h_{j-1}c_{j-1} + 2(h_{j-1} + h_j)c_j + h_j c_{j+1} = \frac{3}{h_j}(a_{j+1} - a_j) - \frac{3}{h_{j-1}}(a_j - a_{j-1})$$

for $j = 1, 2, \ldots, n-1$

Cubic Spline Construction: a System of Linear Equations

$$h_{j-1}c_{j-1}+2\left(h_{j-1}+h_j\right)c_j+h_jc_{j+1}=\frac{3}{h_j}\left(a_{j+1}-a_j\right)-\frac{3}{h_{j-1}}\left(a_j-a_{j-1}\right)$$

for $j=1,2,\ldots,n-1$

1. a_j's are known, h_j are also known; c_j's are the unknowns;
2. can we determine the c_j's? if so, are they unique?

Theorem: Natural Cubic Spline

If $a = x_0 < \ldots < x_n = b$ belong to the domain of f, then f has a unique natural cubic spline interpolant S on the nodes $x_0, \ldots, x_n$.

Theorem: Clamped Cubic Spline

If $a = x_0 < \ldots < x_n = b$ belong to the domain of f and f is differentiable at a and b, then f has a unique clamped cubic spline interpolant S on the nodes $x_0, \ldots, x_n$.

Smoothness

In general, splines are better in data fitting than ordinary polynomials. Why?

1. for the cubic spline, we have continuity of S, S' and S''; the graph appears smooth to the eye (discontinuities in S''' cannot be detected visually);
2. recall that interpolation by ordinary polynomials may exhibit wild oscillations:

   Cubic Spline Smoothness Theorem: for $f \in C^2[a, b]$, $a = x_0 < \ldots < x_n = b$ in the domain of f:

$$\int_a^b \left[S''(x)\right]^2 dx \leq \int_a^b \left[f''(x)\right]^2 dx$$